

Basic Concepts for DV

Ishani Udeshika



Facilitator



Ishani Udeshika



Experience:

Data Scientist at **Zone24x7 pvt Ltd**
Speaker and Trainer

Credentials:

- + M.Sc in Applied Statistics (*reading*)
- + B.Sc in Applied Mathematics and Computing
- + Microsoft Certified Azure Data Scientist

Types of Data

Structured Data

SQL Databases,
Excel File Data



Data that is organized into a fixed format, often in rows and columns, and stored in relational databases.



ID	Name	Age	Country
101	Alice	25	USA
102	Bob	30	Canada

Types of Data

Data that does not follow a strict tabular structure but has some organizational elements (like tags or keys) to make it easier to analyze.

json

```
{  
  "id": 101,  
  "name": "Alice",  
  "age": 25,  
  "skills": ["Python", "SQL"]  
}
```

Semi-structured
Data

XML/JSON Data,
Ex: Emails,
Web Pages



Types of Data

Data that has no predefined structure or format, making it more complex to process and analyze.

Ex:

- Text Files: Articles, books, or emails without any specific structure.
- Images (JPEG, PNG)
- Videos (MP4, AVI)
- Audio files (MP3, WAV)

Unstructured Data

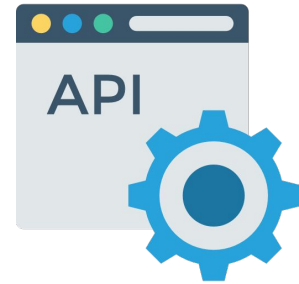
Video, Audio, Image
Data, Documents



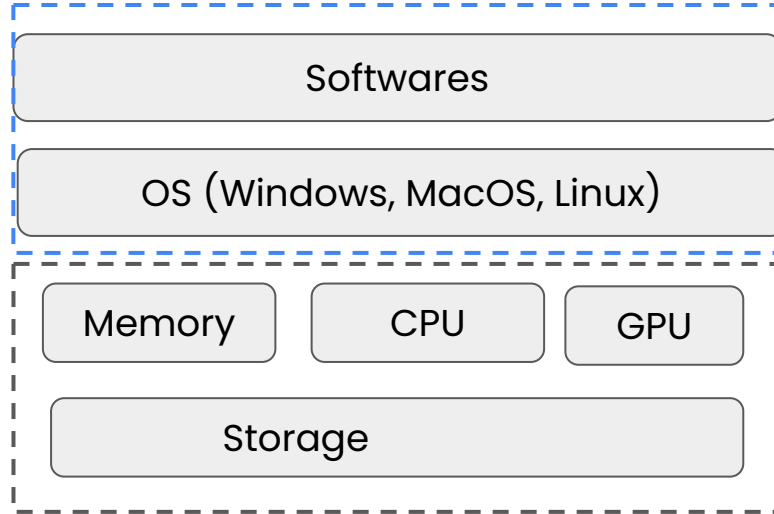
APIs

An API is a set of rules, protocols, and tools that allow different software applications to communicate with each other.

It defines the methods and data formats that applications can use to request and exchange information, enabling seamless interaction and integration between systems.



Node or Computer System



On-premise Vs Cloud Base | Pros and Cons



On-Premises

Pros

- » Full Data Control
- » Full Hard- and Software Control
- » Full Access Reliability
- » No Operating Software Cost
- » Performance
- » Customizations

Cons

- » Maintenance and Acquisition Costs
- » Internal Knowledge
- » Full Responsibility
- » Long-term Reliability



Cloud-Base

Pros

- » Easy Set-up
- » Low Acquisition and Maintenance Costs
- » Flexibility and Scalability
- » Accessibility and Integration
- » Updates and Security
- » Back-up and Data Restore
- » Disaster Recovery

Cons

- » On-going Software Costs
- » Performance Limitations
- » Internet Access

On-Premises

9%

Software Licenses

Customisation & Implementation

Hardware

IT Personnel

Maintenance

Training

Ongoing Costs

- Apply Fixes, Patches, Upgrade
- Downtime
- Performance tuning
- Rewrite customizations
- Rewrite integrations
- Upgrade dependent applications
- Ongoing burden on IT
- Maintain/upgrade hardware
- Maintain/upgrade network
- Maintain/upgrade security
- Maintain/upgrade database

Cloud Computing

68%

Subscription Fee

Implementation,
Customisation &
Training

Ongoing Costs

- Subscription fee

Python Programming for DV

Introduction to Python



Python is a high-level, general-purpose programming language known for its simplicity and readability. It was created by Guido van Rossum and released in 1991.

Because of its emphasis on code readability and a simple syntax, Python is a great language for both inexperienced and seasoned developers.

Python Can Do ..

- Server-side development (web)
- Software application development
- Mathematical and scientific computation
- System scripting and automation



#1

Python

#2

Java

#3

JavaScript

#4

C#

#5

C/C++

#6

PHP

#7

R

#8

TypeScript

#9

Go

#10

Swift

PHP

PYPL index

September 2022

Stack Overflow

Developer Survey 2022

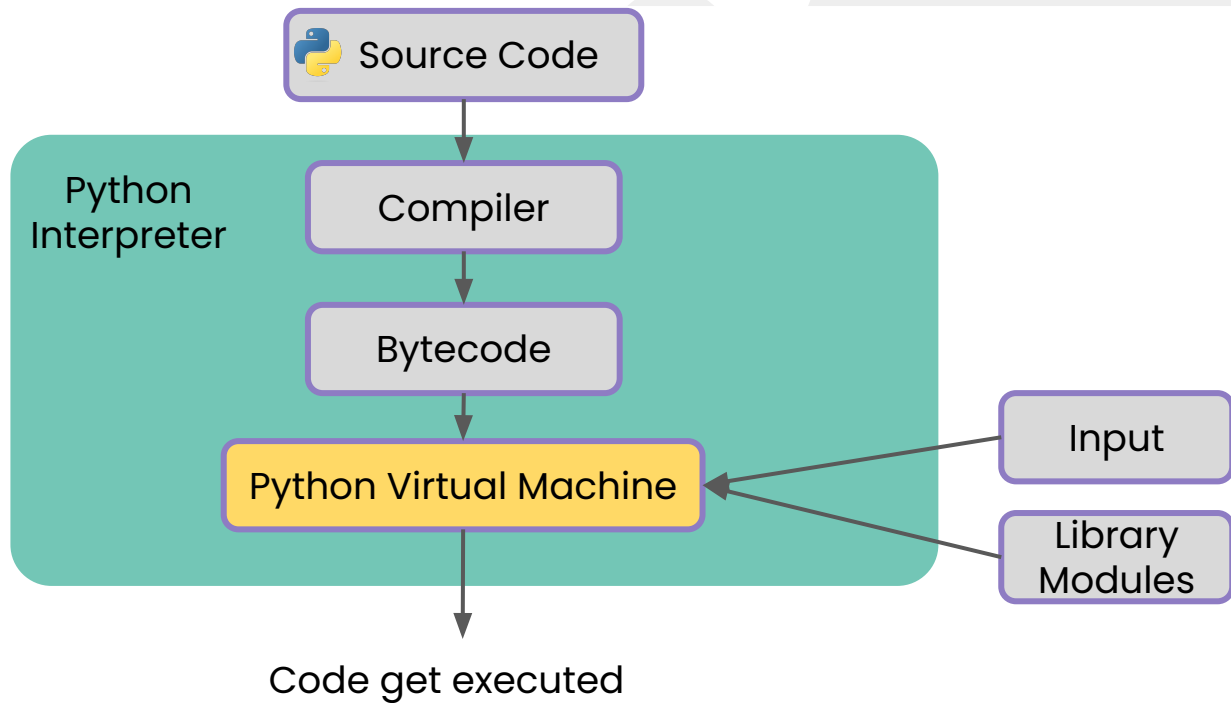


Stackscale

Why Python is popular?

- Python is platform agnostic(Runs on Windows, MacOS, Linux, Raspberry Pi)
- Its an English like language that follow simple syntax
- Fewer lines of code when compare to other programming languages like Java, C++
- Runs on an interpreter, code can be executed as soon as it is written
- Can be treated as procedural and functional way

Python execution



The role of indentation in Python

In Python, indentation plays a crucial role in determining the structure and organization of code blocks. Unlike many other programming languages that use braces or keywords to denote code blocks, Python uses indentation to indicate the grouping and hierarchy of statements within a program.

If the indentation is not correct, you will get an error!

Don't worry, most of the modern IDE supports auto indentation

3 Options to Start Python Journey

1. Install Python and manage the libraries by your own, code with an IDE
2. Install package manager like Anaconda
3. Cloud environment such as Google colab



Comments in Python

Comments can be used to explain your code and make it more human readable. It also helpful to prevent executing your code sections when you testing

```
#This is a comment
print("Hello, World!")

#This is a comment
#written in
#more than just one line
print("Hello, World!")

"""
This is a comment
written in
more than just one line
"""
print("Hello, World!")
```

Variables in Python

The variables are the containers storing data values in Python.

```
x = 5  
y = "John"  
print(x)  
print(y)
```

Data Types: Integer

Hold the numerical values

```
age = 25  
quantity = 10
```

Data Types: Float

A data type that represents decimal numbers with a fractional part

```
pi = 3.14  
price = 9.99
```

Data Types: String

A data type that represents a sequence of characters, such as letters, numbers, and symbols

```
name = "John Doe"  
message = 'Hello, World!'
```

Data Types: Boolean

A data type that represents a logical value, either True or False.

```
is_active = True  
has_permission = False
```

Classroom Activity



Duration: *15 mins*

1. Build your budget



Data Structures

Data Types: List

A data type that represents an ordered collection of elements, which can be of different data types.

```
numbers = [1, 2, 3, 4, 5]  
fruits = ["apple", "banana", "orange"]
```

Data Types: Tuple

A data type that represents an ordered and immutable collection of elements, which can be of different data types.

```
coordinates = (10, 20)  
person = ("John", 25, "john@example.com")
```

Data Types: Set

A data type that represents an unordered collection of unique elements, ensuring no duplicates.

```
unique_numbers = {1, 2, 3, 4, 5}  
unique_characters = {'a', 'b', 'c'}
```

Data Types: Dictionary

A data type that represents a collection of key-value pairs, allowing efficient lookup and retrieval of values based on keys.

```
student = {'name': 'John', 'age': 20, 'grade': 'A'}  
product = {'name': 'Phone', 'price': 499.99, 'quantity': 5}
```

Arithmetic Operators

Arithmetic Operators: Operators used to perform mathematical calculations, such as addition (+), subtraction (-), multiplication (*), division (/), modulo (%), and exponentiation (**).

```
x = 10 + 5 # Addition
y = 10 - 5 # Subtraction
z = 10 * 5 # Multiplication
w = 10 / 5 # Division
mod = 10 % 3 # Modulo (remainder of division)
exp = 2 ** 3 # Exponentiation
```

Comparison Operators

Operators used to compare values and evaluate conditions, such as equal to (`==`), not equal to (`!=`), greater than (`>`), less than (`<`), greater than or equal to (`>=`), and less than or equal to (`<=`).

```
x = 10
y = 5
is_equal = x == y # Equal to
is_not_equal = x != y # Not equal to
is_greater = x > y # Greater than
is_less = x < y # Less than
is_greater_equal = x >= y # Greater than or equal to
is_less_equal = x <= y # Less than or equal to
```

Date Time in Python

- In python there are multiple ways of work with Date and Time. However, Date or Time is not a default data type in python. But we can import the module *DateTime* to work with dates

```
import datetime  
  
now = datetime.datetime.now()  
print(now)
```

Output

2023-08-11 00:55:07.966413

Year Month Day Hour Min Sec Microseconds

Get the current year in Python

- You can extract Individual components

2023-08-11 00:55:07.966413

Year Month Day Hour Min Sec Microseconds

- Return the year from current date time

```
import datetime

now = datetime.datetime.now()
print(now.year)
```

Get the current Month in Python

- You can extract Individual components

2023-08-11 00:55:07.966413

Year Month Day Hour Min Sec Microseconds

- Return the month from current date time

```
import datetime

now = datetime.datetime.now()
print(now.month)
```

Get the current Day in Python

- You can extract Individual components

2023-08-11 00:55:07.966413

Year Month Day Hour Min Sec Microseconds

- Return the day from current date time

```
import datetime

now = datetime.datetime.now()
print(now.day)
```

Get the current year in Python

- You can extract Individual components

2023-08-11 00:55:07.966413

Year Month Day Hour Min Sec Microseconds

- Return the year from current date time

```
import datetime

now = datetime.datetime.now()
print(now.year)
```

Creating a Date

- You can use `datetime()` class to create the `datetime` object which requires three parameters.

```
import datetime  
  
dt = datetime.datetime(2023, 7, 20)  
  
print(dt)
```

-

Format the Output Date

- The return of the datetime can be formatted using `strftime()` method. The output date can format into readable string

```
import datetime  
  
x = datetime.datetime(2018, 6, 1)  
  
print(x.strftime("%B"))
```

-

Format the Output Date cont.

Directive	Description	Example
%a	Weekday, short version	Wed
%A	Weekday, full version	Wednesday
%w	Weekday as a number 0-6, 0 is Sunday	3
%d	Day of month 01-31	31
%b	Month name, short version	Dec
%B	Month name, full version	December
%m	Month as a number 01-12	12
%y	Year, short version, without century	18
%Y	Year, full version	2018
%H	Hour 00-23	17
%I	Hour 00-12	5
%p	AM/PM	PM
%M	Minute 00-59	41
%S	Second 00-59	8
%f	Microsecond 000000-999999	548513
%z	UTC offset	100
%Z	Timezone	CST
%j	Day number of year 001-366	365
%U	Week number of year, Sunday as the first day of week, 00-53	52
%W	Week number of year, Monday as the first day of week, 0-53	52

If and Else Statement

In Python, the "if-else" conditional statement allows you to perform different actions based on a condition being true or false.

```
# Example 1: Checking if a number is positive or negative
num = -7

if num >= 0:
    print("The number is positive or zero.")
else:
    print("The number is negative.")
```

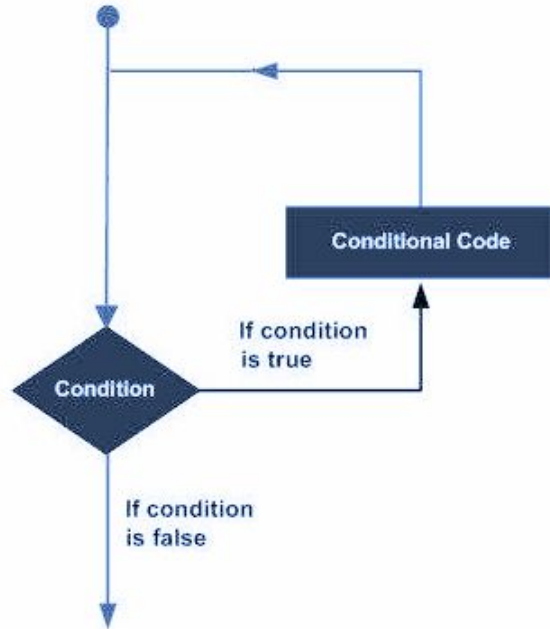
Exercise: Checking if a person is eligible to vote



Repetition Statements

Python Loops

A loop statement allows us to execute a statement or group of statements multiple times.



While Loop

In Python, the "while" loop is used to repeatedly execute a block of code as long as a certain condition remains true.

```
# Example 1: Printing numbers from 1 to 5 using a while loop
num = 1

while num <= 5:
    print(num)
    num += 1
```

Exercise: Calculating the sum of numbers from 1 to 10 using a while loop

For Loop

In Python, the "for" loop is used to iterate over a sequence (such as a list, tuple, or string) or any iterable object, executing a block of code for each element in the sequence.

```
# Example : Printing each character of a string using a for loop
message = "Hello, World!"

for char in message:
    print(char)
```

Exercise: Iterating over a list and performing a repetitive task using a for loop

Loop Control Statements

In Python, for loop control statements "break" allow you to alter the flow of execution within a for loop.

```
# Example : Using break statement to exit a loop early
numbers = [1, 2, 3, 4, 5]

for num in numbers:
    if num == 3:
        break
    print(num)
```

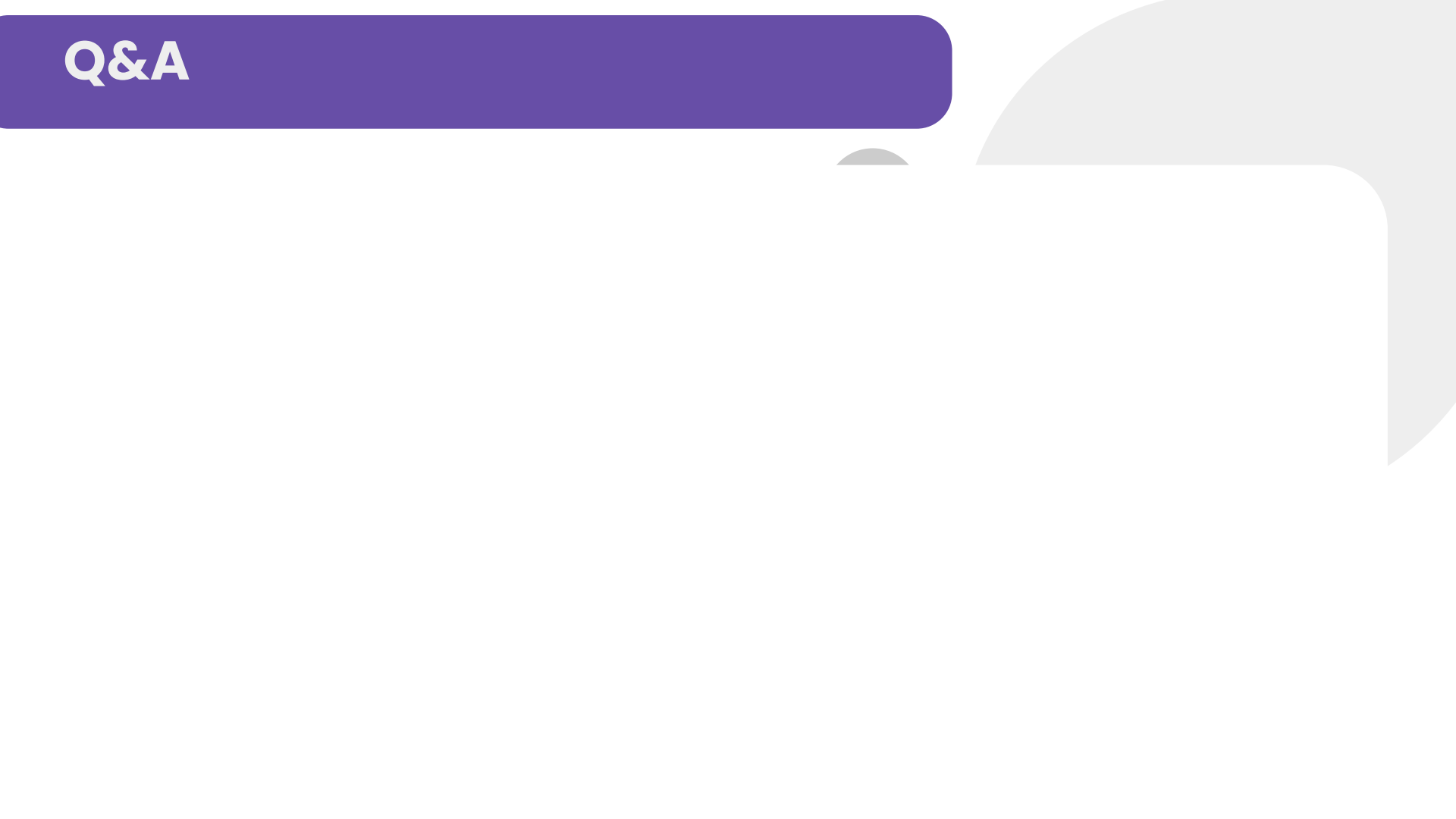
Loop Control Statements

In Python, for loop control statements "continue" allow you to alter the flow of execution within a for loop.

```
# Example : Using continue statement to skip an iteration
numbers = [1, 2, 3, 4, 5]

for num in numbers:
    if num == 3:
        continue
    print(num)
```

Q&A



Thank You